

Módulo 5 – Capítulo 03 – Sección 01

Por qué usar un framework de orquestación: abstracción, cadenas y composición

Implementar un chatbot simple que toma el input del usuario, construye un prompt, llama al LLM y devuelve la respuesta requiere menos de 20 líneas de Python con el SDK puro del proveedor. Implementar un sistema que recupera documentos relevantes de múltiples fuentes, los incorpora al contexto, llama al LLM con historial conversacional, parsea la respuesta estructurada, invoca herramientas externas según el resultado, gestiona el estado de la conversación entre sesiones, y emite trazas de observabilidad en cada paso puede requerir centenares de líneas de código con gestión manual de estado, serialización, propagación de errores y manejo de casos borde que son difíciles de anticipar desde el inicio. Los frameworks de orquestación emergen como respuesta a esta complejidad creciente: ofrecen abstracciones que encapsulan los patrones recurrentes de los flujos de IA, reduciendo el código repetitivo y haciendo los flujos complejos legibles como composición de componentes.

La abstracción central que ofrecen estos frameworks es la idea de "cadena" o "pipeline": una secuencia de pasos donde la salida de cada componente es la entrada del siguiente, con manejo automático de tipos, serialización, propagación de errores y observabilidad integrada. LangChain Expression Language (LCEL) modela esta composición con el operador `|`: `chain` `= prompt_template | llm | output_parser` crea un objeto Runnable ejecutable donde el sistema gestiona internamente el formateo del prompt, la llamada al LLM, el parseo de la respuesta, y potencialmente el retry si el parseo falla. Esta abstracción declarativa hace que el código exprese la intención del flujo —qué pasos componen el pipeline— sin mezclarlo con los detalles de implementación de cada paso.

La intercambiabilidad de componentes es otro beneficio concreto. Una cadena LCEL que usa `ChatOpenAI` puede cambiar al modelo de Anthropic reemplazando el componente LLM por `ChatAnthropic` sin modificar el prompt template ni el parser. En producción, esta propiedad permite hacer A/B testing de proveedores, fallback entre proveedores en caso de degradación, y migración de modelo sin reescribir el pipeline. El mismo principio aplica a los otros

componentes: el vector store puede cambiar de Chroma a pgvector, el parser de `StrOutputParser` a `PydanticOutputParser`, sin que el resto del pipeline necesite modificarse.

La observabilidad integrada es quizás el argumento más práctico para adoptar un framework en proyectos serios. LangChain emite callbacks en cada paso de la cadena que herramientas como LangSmith o Langfuse capturan automáticamente: cada llamada al LLM, cada consulta al vector store, cada invocación de herramienta queda registrada con su timing, los tokens consumidos y los datos de entrada y salida. Instrumentar este nivel de observabilidad manualmente —sin framework— requiere añadir código de logging en cada función, propagar context IDs, y gestionar la jerarquía de spans: trabajo significativo que el framework hace gratis cuando se tiene la integración configurada.

Conceptos clave de la orquestación

- **Cadena (chain):** secuencia de componentes que transforma una entrada en una salida mediante llamadas a LLMs, herramientas y transformaciones de datos; es la unidad de composición fundamental y puede anidarse dentro de cadenas más complejas.
- **Runnable interface (LCEL):** protocolo común que todos los componentes de LangChain implementan con métodos `invoke()`, `stream()`, `batch()` y sus variantes async, haciendo que sean conectables con `|` independientemente de su tipo interno.
- **Memory y state management:** abstracciones para mantener historial conversacional en memoria, Redis, bases de datos SQL o vectoriales, con compresión automática del historial cuando el contexto se aproxima al límite del modelo.
- **Composición declarativa vs imperativa:** declarar el flujo como composición de componentes facilita la inspección con herramientas de debugging, el testing por partes con mocks de componentes individuales, y la modificación sin tocar la lógica de integración de los demás pasos.
- **Observabilidad integrada:** LangChain emite eventos en cada paso que herramientas como LangSmith capturan sin modificar el código, proveyendo trazas jerarquizadas de cada request con latencia, tokens y datos de cada componente.

Un framework de orquestación justifica su adopción cuando el beneficio en legibilidad, mantenibilidad y reutilización de componentes supera el costo de aprender su modelo mental y gestionar sus actualizaciones frecuentes de API. Las secciones siguientes examinan los tres frameworks principales —LangChain, LlamaIndex y DSPy— con sus fortalezas específicas, los casos de uso donde cada uno brilla, y los criterios objetivos para elegir entre ellos.

Principio rector: Un framework de orquestación justifica su adopción cuando el beneficio en legibilidad, mantenibilidad y reutilización de componentes supera el costo de aprender su modelo mental y gestionar sus actualizaciones; el criterio de adopción debe ser empírico, no aspiracional.

Módulo 5 – Capítulo 03 – Sección 02

LangChain y LangGraph: cadenas, runnables, agentes con estado

LangChain es el framework de orquestación de IA más adoptado en la industria, con más de 90.000 estrellas en GitHub y un ecosistema de integraciones que cubre más de 200 vectorstores, herramientas y proveedores de LLM. Su arquitectura actual se organiza en cuatro paquetes con responsabilidades separadas: `langchain-core` contiene las interfaces base y el protocolo LCEL; `langchain` provee las cadenas y agentes de alto nivel; `langchain-community` concentra las integraciones de terceros mantenidas por la comunidad; y los paquetes específicos de proveedor — `langchain-openai`, `langchain-anthropic`, `langchain-google-genai` — encapsulan las integraciones con los SDKs nativos de cada uno. Esta modularidad permite instalar solo las dependencias que el proyecto necesita, en lugar del monolito que fue en sus versiones iniciales.

La columna vertebral del framework en su versión moderna es LCEL, LangChain Expression Language. LCEL implementa la interfaz `Runnable` con métodos `invoke(input)`, `stream(input)`, `batch(inputs)` y sus variantes async `ainvoke`, `astream`, `abatch`. El operador `|` conecta componentes que implementan `Runnable` en una secuencia: `chain = ChatPromptTemplate.from_messages([...]) | ChatAnthropic(model="claude-3-5-sonnet-20241022") | PydanticOutputParser(pydantic_object=MySchema)`. Cuando se llama `chain.invoke({"question": "..."})`, el framework gestiona internamente el formateo del template con las variables, la llamada al LLM con los parámetros configurados, el parseo de la respuesta según el schema Pydantic, y el retry con `OutputFixingParser` si el parseo falla. Cada paso recibe la salida tipada del paso anterior, lo que el framework verifica en tiempo de ejecución.

`RunnableParallel` y `RunnableBranch` extienden el modelo lineal hacia flujos más complejos. `RunnableParallel` ejecuta múltiples runnables en paralelo con asyncio y combina sus salidas en un diccionario: útil para llamar simultáneamente a múltiples recuperadores de datos o ejecutar el mismo query sobre diferentes modelos para comparación. `RunnableBranch` implementa routing condicional: evalúa una condición sobre la entrada y enruta hacia uno de

various runnables según el resultado, permitiendo flujos donde diferentes tipos de query reciben procesamiento diferenciado sin código imperativo explícito.

LangGraph es la extensión de LangChain para flujos cíclicos y agentes con estado persistente —el mecanismo de implementación para los agentes con memoria que el Módulo 4 describió arquitectónicamente. Donde LCEL modela flujos como cadenas lineales o DAGs acíclicos, LangGraph modela flujos como grafos dirigidos con nodos, aristas condicionales y estado compartido. Un grafo de LangGraph se define con `StateGraph(MyState)`, donde `MyState` es un TypedDict que representa el estado compartido entre nodos. Los nodos son funciones Python que reciben el estado completo y devuelven actualizaciones parciales: `def process_query(state: MyState) -> dict: ...`. Las aristas condicionales se añaden con `add_conditional_edges(node, routing_function, {"option_a": "node_a", "option_b": "node_b"})`, donde la función de routing decide hacia qué nodo avanzar según el estado actual. El `MemorySaver` persiste el estado entre ejecuciones del grafo en memoria o en una base de datos, haciendo posible los agentes con historial de sesión que retienen contexto entre sesiones separadas.

La observabilidad de LangGraph se integra de forma nativa con LangSmith cuando `LANGCHAIN_TRACING_V2=true` está configurado: cada nodo del grafo aparece como un span hijo en la traza jerárquica, mostrando el estado antes y después de cada nodo, los modelos llamados, y el routing condicional tomado en cada arista, lo que transforma el debugging de agentes complejos de una tarea artesanal en una tarea visual.

Componentes principales de LangChain y LangGraph

- `ChatPromptTemplate.from_messages()`: define la estructura del prompt con slots para variables `{variable}` y para el historial conversacional `MessagesPlaceholder`; soporta formateo condicional de mensajes y reutilización entre cadenas.
- **Wrappers de LLM** (`ChatOpenAI`, `ChatAnthropic`): implementan la interfaz `Runnable` y encapsulan el SDK del proveedor; son intercambiables dentro de una cadena LCEL sin modificar los demás componentes.
- `PydanticOutputParser` + `with_structured_output()`: el método `with_structured_output(MySchema)` disponible en los wrappers modernos usa tool use o structured outputs del proveedor para garantizar JSON válido según el schema sin necesidad de parser manual.
- `StateGraph` (**LangGraph**): el grafo de estado donde cada nodo es una función Python con acceso al estado completo; `add_node()`, `add_edge()` y `add_conditional_edges()`

construyen la topología del grafo de forma declarativa.

- **MemorySaver (LangGraph):** checkpointing que persiste el estado del grafo por `thread_id`, habilitando agentes con memoria de sesión; soportable con backends de base de datos para persistencia durable entre reinicios del servicio.

Nota del Arquitecto: *LangChain tiene historial de breaking changes significativos entre versiones mayores (v0.1 a v0.2, v0.2 a v0.3). La práctica de anclar la versión exacta en `requirements.txt` (`langchain==0.3.7`, `langchain-anthropic==0.3.2`) y revisar el changelog antes de actualizar no es paranoia sino experiencia acumulada de proyectos que sufrieron regresiones tras actualizaciones automáticas de dependencias. LangGraph, por ser una extensión más joven, tiene aún mayor velocidad de cambio de API.*

La decisión de usar LangGraph en lugar de LCEL puro depende de un criterio concreto: si el flujo del agente necesita ciclos (llamar al LLM, evaluar el resultado, y volver a llamar al LLM con el resultado corregido) o ramificación condicional compleja que depende del estado acumulado entre pasos, LangGraph es la abstracción correcta. Si el flujo es una secuencia lineal de pasos sin ciclos, LCEL es suficiente y más simple de debuggear.

Para recordar: LangGraph implementa los agentes con estado que el Módulo 4 describió arquitectónicamente: el modelo de nodos-aristas-estado transforma las cadenas lineales de LCEL en grafos dirigidos con ciclos de reflexión, memoria persistente entre sesiones y routing condicional que habilita el comportamiento autónomo de los agentes.

Módulo 5 – Capítulo 03 – Sección 03

LlamaIndex: orientado a RAG, índices, query engines y conectores de datos

Mientras LangChain ofrece una abstracción general para componer flujos de múltiples pasos, LlamaIndex parte de un supuesto diferente: el problema central de la mayoría de las aplicaciones de IA empresarial no es la orquestación de herramientas sino la gestión de datos. La pregunta que LlamaIndex responde es: ¿cómo conectar los LLMs con los datos propios de la organización de forma eficiente, escalable y actualizable? Su abstracción central no es la cadena sino el índice, una estructura que organiza documentos —PDFs, páginas de Notion, tablas SQL, tickets de Jira, conversaciones de Slack— para recuperación eficiente mediante búsqueda semántica, por palabras clave o híbrida según la naturaleza de la query.

El flujo canónico de LlamaIndex tiene tres etapas bien diferenciadas. La etapa de ingesta carga los documentos desde su fuente original: `SimpleDirectoryReader` lee PDFs, DOCX, TXT, HTML y Markdown de un directorio local con detección automática de tipo; los conectores específicos —`NotionPageReader`, `SlackReader`, `DatabaseReader`— se conectan a APIs externas para cargar datos con metadata asociada. Cada documento cargado es un objeto `Document` con texto y metadata extensible (fuente, fecha, autor, URL) que se propaga a los nodos generados en la etapa siguiente. La etapa de indexación transforma los documentos en embeddings y los almacena en el vector store configurado: `VectorStoreIndex.from_documents(documents, storage_context=StorageContext.from_defaults(vector_store=qdrant_store))` genera automáticamente los embeddings con el modelo configurado —`text-embedding-3-small`, `embed-multilingual-v3.0` de Cohere, o cualquier otro— y los persiste en el vector store. La etapa de consulta usa un `QueryEngine` que orquesta la recuperación y síntesis: dado un query en lenguaje natural, el engine recupera los `top_k` nodos más semánticamente relevantes, construye el prompt de síntesis incluyendo el contexto recuperado, y devuelve un objeto `Response` con la respuesta y las fuentes citadas con sus scores de relevancia.

La abstracción `NodeParser` controla cómo los documentos se dividen en chunks para la indexación, con consecuencias directas sobre la calidad de la recuperación.

`SentenceSplitter` divide por oraciones respetando límites semánticos; `SemanticSplitter` agrupa oraciones por similitud semántica usando embeddings, creando chunks con coherencia conceptual en lugar de simple límite de caracteres; `MarkdownNodeParser` entiende la estructura de encabezados de los documentos Markdown y los usa como límites de división, preservando la jerarquía del documento en los chunks. La elección del parser tiene impacto medible en las métricas de faithfulness y context recall: chunks semánticamente coherentes se recuperan con mayor precisión que chunks divididos por longitud fija.

Los query engines avanzados de LlamaIndex abordan casos de uso que van más allá de la recuperación simple. `RouterQueryEngine` selecciona dinámicamente entre múltiples índices según la naturaleza de la pregunta, enrutando preguntas sobre ventas al índice de CRM y preguntas sobre soporte al índice de tickets. `SubQuestionQueryEngine` descompone preguntas complejas en subpreguntas independientes sobre diferentes fuentes de datos, las ejecuta en paralelo, y sintetiza las respuestas en una respuesta cohesiva. `IngestionPipeline` define un pipeline de transformaciones sobre los documentos que puede ejecutarse de forma incremental, procesando solo los documentos nuevos o modificados desde la última ejecución, sin reindexar el corpus completo.

Componentes principales de LlamaIndex

- `SimpleDirectoryReader` : carga documentos de un directorio con detección automática de tipo; acepta `required_exts=[".pdf", ".md"]` para filtrar por extensión; devuelve objetos `Document` con metadata de nombre de archivo, ruta y tipo de contenido.
- `VectorStoreIndex` : indexa documentos generando embeddings y almacenándolos en el vector store configurado; soporta Qdrant, Pinecone, Chroma, pgvector y Weaviate; con `persist_dir` persiste el índice en disco para no regenerarlo en cada inicio.
- `QueryEngine.query()` : recibe una pregunta en lenguaje natural y devuelve un `Response` con `response.response` (texto) y `response.source_nodes` (lista de nodos recuperados con scores); el argumento `similarity_top_k=5` controla cuántos nodos se recuperan.
- `RetrieverQueryEngine` con `SimilarityPostprocessor` : filtra los nodos recuperados por umbral de score de similitud coseno antes de pasarlos al modelo de síntesis, reduciendo ruido cuando el recuperador trae documentos marginalmente relevantes.
- `IngestionPipeline` : define transformaciones secuenciales sobre documentos (chunking, extracción de metadata, generación de embeddings) ejecutables de forma incremental sobre nuevos documentos sin reindexar el corpus existente.

LlamaIndex brilla cuando los datos son la fuente principal de variabilidad y el patrón dominante es recuperar-sintetizar. La sección siguiente presenta DSPy, un framework con un paradigma radicalmente diferente: en lugar de escribir los prompts manualmente, los optimiza automáticamente mediante evaluación.

Principio rector: LlamaIndex es la abstracción correcta cuando la complejidad del sistema reside en la gestión de datos —múltiples fuentes, chunking semánticamente correcto, recuperación de alta calidad— y el patrón dominante es RAG; para flujos que priorizan la lógica de orquestación sobre los datos, LangChain o implementación directa son más adecuados.

Módulo 5 – Capítulo 03 – Sección 04

DSPy: optimización declarativa de prompts

LangChain y LlamaIndex comparten un supuesto fundamental: el ingeniero escribe los prompts manualmente. El ingeniero decide las instrucciones del sistema, los ejemplos few-shot, el formato de salida solicitado y la cadena de razonamiento que el modelo debe seguir. DSPy —Declarative Self-improving Python— parte de un supuesto diferente: ¿por qué escribir prompts manualmente cuando es posible que un sistema los optimice automáticamente, dado un conjunto de ejemplos de entrenamiento y una función de evaluación? Esta pregunta define un paradigma nuevo que no es una variante de LangChain ni de LlamaIndex, sino una alternativa conceptualmente distinta para un problema específico: sistemas donde el prompt ideal no es obvio a priori y donde la evaluación automática puede guiar la búsqueda.

El modelo mental de DSPy tiene tres componentes. Las **Signatures** son la unidad de abstracción fundamental: definen la interfaz del módulo como un mapeo de campos de entrada a campos de salida con tipos y descripciones. `class QASignature(dspy.Signature):`
`"Answer a question given context."; context: str = dspy.InputField(); question: str`
`= dspy.InputField(); answer: str = dspy.OutputField()`. La Signature declara qué hace el módulo sin especificar cómo hacerlo: es la interfaz, no la implementación. Los **Módulos** son los componentes de construcción que implementan la Signature usando diferentes estrategias de prompting. `dspy.Predict(QASignature)` genera una respuesta directa; `dspy.ChainOfThought(QASignature)` instruye al modelo a razonar paso a paso antes de responder; `dspy.ReAct(QASignature, tools=[...])` implementa el patrón de agente con herramientas. La diferencia con LangChain es que ninguno de estos módulos tiene un prompt hardcodeado: el prompt es una variable que el sistema puede modificar.

Los **Teleprompters** (u Optimizadores) son el mecanismo que diferencia a DSPy de todos los demás frameworks. `dspy.teleprompt.BootstrapFewShot` toma el programa DSPy, un conjunto de ejemplos de entrenamiento, y una función de evaluación; ejecuta el programa sobre los ejemplos, evalúa los resultados, selecciona los mejores few-shot examples automáticamente, y los inyecta en los prompts de cada módulo. `dspy.teleprompt.MIPROv2` va más allá: optimiza simultáneamente las instrucciones del sistema y los few-shot examples

usando un proceso de búsqueda en el espacio de prompts guiado por la métrica de evaluación. El resultado es un programa compilado donde cada módulo tiene los prompts optimizados para maximizar la métrica definida —faithfulness, accuracy, F1 o cualquier función custom— sobre el conjunto de validación.

El caso de uso donde DSPy tiene mayor ventaja sobre los enfoques manuales es cuando el prompt ideal es difícil de especificar por intuición: tareas de razonamiento complejo en dominios especializados, sistemas donde los few-shot examples mejoran significativamente el rendimiento pero elegirlos manualmente es costoso, y sistemas donde múltiples módulos encadenados necesitan ser co-optimizados porque el rendimiento de cada uno depende de los demás. La relación entre DSPy y el sistema de evaluación del capítulo 7 es directa: el dataset de evaluación curado es la materia prima del optimizador de DSPy; mientras mejor sea el dataset, más efectiva es la optimización.

DSPy tiene limitaciones conocidas que definen su alcance actual. El overhead de runtime del proceso de optimización puede ser significativo —del orden de decenas a centenares de llamadas al LLM para optimizar un programa simple—, por lo que la optimización se hace offline y el programa compilado se carga para inferencia. La reproducibilidad de la optimización no está garantizada porque el proceso depende de muestreo estocástico sobre el dataset. La compatibilidad con modelos open-source locales es funcional pero con mayor variabilidad en los resultados comparada con modelos de API como Claude o GPT-4.

Componentes principales de DSPy

- **`dspy.Signature`** : contrato de entrada/salida del módulo; campos con `dspy.InputField()` y `dspy.OutputField()` más docstring descriptivo; no especifica el prompt, solo la interfaz.
- **`dspy.ChainOfThought`** : wrapper que añade razonamiento paso a paso antes del campo de salida; el proceso de razonamiento es un campo intermedio que el optimizador puede modificar para mejorar la calidad.
- **`dspy.teleprompt.BootstrapFewShot`** : optimizador que selecciona automáticamente few-shot examples del training set que maximizan la métrica de evaluación; requiere un conjunto de entrenamiento con pares (input, expected_output) y una función de evaluación.
- **`dspy.teleprompt.MIPROv2`** : optimizador de instrucciones y few-shot examples conjuntamente; utiliza un proceso de búsqueda en el espacio de prompts con evaluaciones

múltiples; más poderoso que BootstrapFewShot pero con mayor costo computacional de optimización.

- **Compilación y serialización:** `compiled_program = optimizer.compile(program, trainset=trainset, metric=metric)` produce un programa con prompts optimizados; `compiled_program.save("program.json")` persiste los prompts para carga sin re-optimizar en cada inicio.

La sección siguiente compara los tres frameworks directamente en términos de fortalezas, debilidades y criterios de selección, con un criterio cuantitativo de decisión basado en la complejidad del flujo y las características del caso de uso.

Buena práctica: DSPy es la elección correcta cuando el prompt ideal no es obvio por intuición y la evaluación automática puede guiar la búsqueda; el prerequisite es tener un dataset de entrenamiento de calidad suficiente y una función de evaluación que correlacione con la calidad real del sistema —los mismos activos que el sistema de evaluación continua del capítulo 7 provee.

Módulo 5 – Capítulo 03 – Sección 05

Comparación y decisión: LangChain, LlamaIndex, DSPy o implementación directa

Elegir entre LangChain, LlamaIndex, DSPy o llamadas directas al SDK del proveedor no es una decisión de preferencia personal ni de popularidad relativa en GitHub: es una decisión de ingeniería basada en el análisis objetivo de los requisitos del caso de uso, la complejidad del flujo, el perfil del equipo, y el costo de mantenimiento de la abstracción a lo largo del ciclo de vida del sistema. El error más frecuente es adoptar prematuramente el framework más popular sin evaluar si las abstracciones que ofrece resuelven los problemas específicos del proyecto, resultando en código que usa el 5% de las capacidades del framework mientras carga con el 100% de sus dependencias y complejidad.

La implementación directa con el SDK puro del proveedor es el punto de referencia contra el cual medir cualquier framework. Para flujos simples —una llamada al LLM por request, respuesta devuelta sin transformación compleja—, el SDK directo es la solución más controlable, predecible y debuggeable: el stack de llamadas completo está en código propio, los errores tienen trazas directas sin indirecciones de framework, y cualquier desarrollador Python puede leer y modificar el código sin conocer abstracciones específicas. El criterio cuantitativo es claro: si el flujo completo cabe en menos de 100 líneas limpias con el SDK puro, incluyendo el manejo de errores y el logging, la implementación directa es preferible.

LangChain es la opción cuando la complejidad del flujo supera ese umbral y el problema dominante es la orquestación: múltiples herramientas que el agente puede invocar, composición de llamadas al LLM con recuperación de datos y transformaciones intermedias, o flujos con ramificación condicional basada en el resultado de pasos anteriores. LangGraph, como extensión de LangChain, es específicamente la herramienta correcta cuando el flujo necesita ciclos —un agente que llama al LLM, evalúa si la respuesta es suficiente, y vuelve a llamar con información adicional— o cuando el estado debe persistir entre sesiones del usuario. La fortaleza de LangChain reside en su ecosistema: 200+ integraciones disponibles, documentación extensa y LangSmith para observabilidad nativa.

LlamaIndex es la opción dominante cuando el problema central es la gestión de datos: múltiples fuentes heterogéneas (PDFs, bases de datos, APIs), chunking semánticamente correcto, y pipelines de recuperación avanzados como `RouterQueryEngine` o `SubQuestionQueryEngine`. Su ecosistema de 150+ conectores de datos elimina semanas de trabajo de integración que la implementación directa requeriría. Para sistemas RAG sobre más de 10.000 documentos con múltiples fuentes, LlamaIndex ahorra un mes de desarrollo de la capa de ingesta, indexación y recuperación.

DSPy es la opción cuando el prompt ideal no puede determinarse por ingeniería manual y la tarea tiene un dataset de entrenamiento suficiente con una función de evaluación bien definida. La condición práctica para adoptar DSPy es que el equipo tenga disponibles más de 50 ejemplos de entrenamiento anotados y una función de evaluación que correlacione con la calidad real —exactamente los activos que el sistema de evaluación del capítulo 7 produce. DSPy no reemplaza a LangChain ni a LlamaIndex en la capa de orquestación; puede combinarse con ambos, usando DSPy para optimizar los prompts de módulos dentro de una cadena LangChain o de un query engine de LlamaIndex.

Tabla de decisión por criterio

La siguiente tabla cruza los factores más determinantes con la recomendación de cada alternativa:

Criterio	Implementación directa	LangChain / LangGraph	LlamaIndex	DSPy
Pasos en el pipeline	1-2 pasos	3+ con ramificación o ciclos	Recuperación + síntesis	Cualquier flujo con evaluación
Fuentes de datos	Una fuente simple	No determinante	Múltiples fuentes	No determinante
Agentes con estado	No aplica	LangGraph	No aplica	No aplica
Prompt óptimo conocido	Sí	Sí	Sí	No
Dataset de entrenamiento	No necesario	No necesario	No necesario	Necesario (>50 ejemplos)
Debuggabilidad	Máxima	Media (mejorada con LangSmith)	Media	Baja durante optimización
Overhead de dependencias	Mínimo	Alto (100-200 deps transitivas)	Alto	Medio

Señales de que un framework no está justificado

- **Flujo de un solo paso:** una sola llamada al LLM por request sin ramificación ni composición hace que cualquier framework sea overhead puro.
- **Equipo pequeño sin experiencia previa:** en equipos de 1-2 personas el costo de aprendizaje del framework puede superar su beneficio durante los primeros tres a seis meses del proyecto.
- **Alta necesidad de debuggabilidad:** cuando los bugs son costosos de reproducir, tener el stack completo en código propio es la ventaja más valiosa frente a cualquier framework.
- **Flujo estable y bien entendido:** si el flujo no cambiará significativamente en los próximos seis a doce meses, la implementación directa bien estructurada es más mantenible que una cadena de componentes de terceros.

El criterio de decisión más práctico en la industria es el "test de la implementación directa": antes de adoptar un framework, intentar implementar el flujo con el SDK puro y estimar cuántas líneas requiere. Si el resultado es manejable, la implementación directa es preferible.

Si la complejidad que emerge —manejo del historial, múltiples fuentes de datos, ciclos de agente— genera código difícil de mantener, el framework está justificado.

Idea central: La decisión de adoptar un framework debe evaluarse con la misma rigurosidad que cualquier dependencia de terceros: ¿qué problema específico resuelve que no se puede resolver en menos código propio? La respuesta honesta a esa pregunta determina si el framework aporta o añade complejidad accidental.

Módulo 5 – Capítulo 03 – Sección 06

Cierre: criterios para elegir entre orquestación directa y frameworks

La elección entre frameworks de orquestación e implementación directa no es ideológica sino pragmática, y el equipo que la toma con criterios claros evita dos errores simétricos igualmente costosos: adoptar prematuramente todo el stack posible por aspiración a la sofisticación técnica, o rechazar los frameworks por dogma de "código propio es mejor" cuando la complejidad del flujo justifica la abstracción. Ambos errores tienen el mismo resultado: sistemas más difíciles de mantener de lo necesario, por razones opuestas.

El criterio más útil en la práctica es el "test de la implementación directa": antes de comprometerse con un framework, implementar el flujo completo con el SDK puro del proveedor, incluyendo el manejo de errores, el logging, la gestión del historial y los casos borde que emergen del análisis de requisitos. Si el resultado cabe en menos de 150 líneas de código claro y el equipo puede entenderlo de un vistazo, la implementación directa es preferible y el framework añadiría complejidad sin justificación. Si la implementación directa crece hacia 500 o más líneas con lógica de estado compleja, múltiples fuentes de datos que necesitan indexación, o ciclos de agente con herramientas, el framework está ganando su costo.

La estrategia más sólida en proyectos de IA maduros es la composición cuidadosa de capas: el SDK directamente para las llamadas al LLM simples donde el control granular importa, LlamaIndex solo para la capa de recuperación vectorial cuando el caso de uso es RAG intensivo con múltiples fuentes, LangGraph cuando la lógica del agente necesita ciclos y estado persistente, y DSPy cuando el sistema de evaluación tiene los datasets necesarios para guiar la optimización automática de prompts. Esta composición evita el "todo o nada" de comprometerse con un único framework para toda la aplicación, manteniendo el control donde importa y usando la abstracción donde ahorra trabajo real.

El mantenimiento a largo plazo es el criterio que los análisis de adopción inicial más frecuentemente subestiman. Los frameworks de orquestación tienen velocidad de cambio de

API alta: LangChain realizó breaking changes significativos entre v0.1, v0.2 y v0.3, y cada actualización requirió refactorización del código que usaba las interfaces modificadas. Los sistemas contruidos sobre implementación directa con el SDK del proveedor no se ven afectados por estos cambios de framework; el proveedor de LLM tiene contratos de estabilidad de API más sólidos que los frameworks de la comunidad open source. Este trade-off —velocidad de desarrollo vs estabilidad a largo plazo— es una dimensión de la decisión que debe incorporarse explícitamente.

Criterios de decisión sintetizados

- **Complejidad del flujo:** 1-2 pasos lineales favorecen la implementación directa; 3 o más pasos con ramificación, paralelismo o ciclos favorecen un framework con el modelo correcto para ese patrón.
- **Volumen de datos para recuperación:** para RAG sobre más de 10.000 documentos con múltiples fuentes heterogéneas, LlamaIndex elimina semanas de desarrollo de ingesta, chunking e indexación.
- **Velocidad de cambio del flujo:** flujos que evolucionan rápidamente se benefician de la modularidad del framework para cambiar componentes; flujos estables se benefician de la predictibilidad de la implementación directa.
- **Disponibilidad de datos de entrenamiento:** si existen más de 50 ejemplos etiquetados y una función de evaluación definida, DSPy es viable; sin ellos, la implementación manual de prompts es la única opción.
- **Madurez del equipo en el framework específico:** equipos con experiencia previa en LangChain o LlamaIndex obtienen productividad real desde el primer sprint; equipos que aprenden el framework en paralelo con el proyecto pagan un costo de learning curve que puede superar el beneficio durante los primeros meses.

Nota del Arquitecto: La estrategia más efectiva que he visto en equipos productivos es: comenzar con el SDK puro para los primeros sprints de validación del caso de uso, adoptar LlamaIndex cuando el volumen de documentos hace que la indexación manual sea costosa, y adoptar LangGraph cuando la lógica del agente requiere ciclos que el LCEL lineal no puede expresar. Este orden de adopción —guiado por evidencia de necesidad real, no por plan previo— resulta en sistemas que tienen el mínimo de dependencias necesarias y el máximo de control sobre lo que importa.

La elección correcta entre frameworks es la que el equipo puede mantener, que encaja con los requisitos del caso de uso, y que fue evaluada empíricamente antes de comprometerse con ella. Los capítulos siguientes asumen que esta elección ha sido tomada y se centran en las capas de ingeniería que aplican independientemente del framework: los patrones de integración con sistemas existentes, el testing, el CI/CD, la evaluación de calidad y la observabilidad.

"The best tool is the one you know how to use." — Adaptado de Donald Knuth. En AI Engineering, el framework que el equipo domina y que encaja con los requisitos del caso de uso siempre superará en productividad al framework más popular que el equipo no conoce bien o que resuelve un problema que el proyecto no tiene.